

Two-Stage Rocket Simulation

Physics and Parameter

Physical and Numerical Parameters (SI) :

$$g = 9.81 \text{ m/s}^2$$

$$\rho_0 = 1.225 \text{ kg/m}^3$$

$$H = 8400 \text{ m}$$

$$\text{Drag coefficient} = 0.6$$

$$A = 0.01 \text{ m}^2$$

Dynamics :

$$h' = v \quad v' = \frac{(T(t) - D(v, h))}{m(t) - g}$$

Air density at altitude h :

$$\rho(h) = \rho_0 \cdot \exp\left(\frac{-h}{H}\right)$$

Aerodynamic Drag Force :

$$D = \frac{1}{2} \cdot \rho(h) \cdot \text{Drag coefficient} \cdot A \cdot v \cdot |v|$$

Thrust profile (auto-ignition) :

Stage 1: constant thrust T_1 for 8s

Stage 2: ignition when $q \leq q_{\max}$, $v > 0$, and $\Delta t \geq t_{\min}$

($\Delta t = 0.8\text{s}$ delay to simulate separation)

Stage 2: constant thrust T_2 for burn 6s

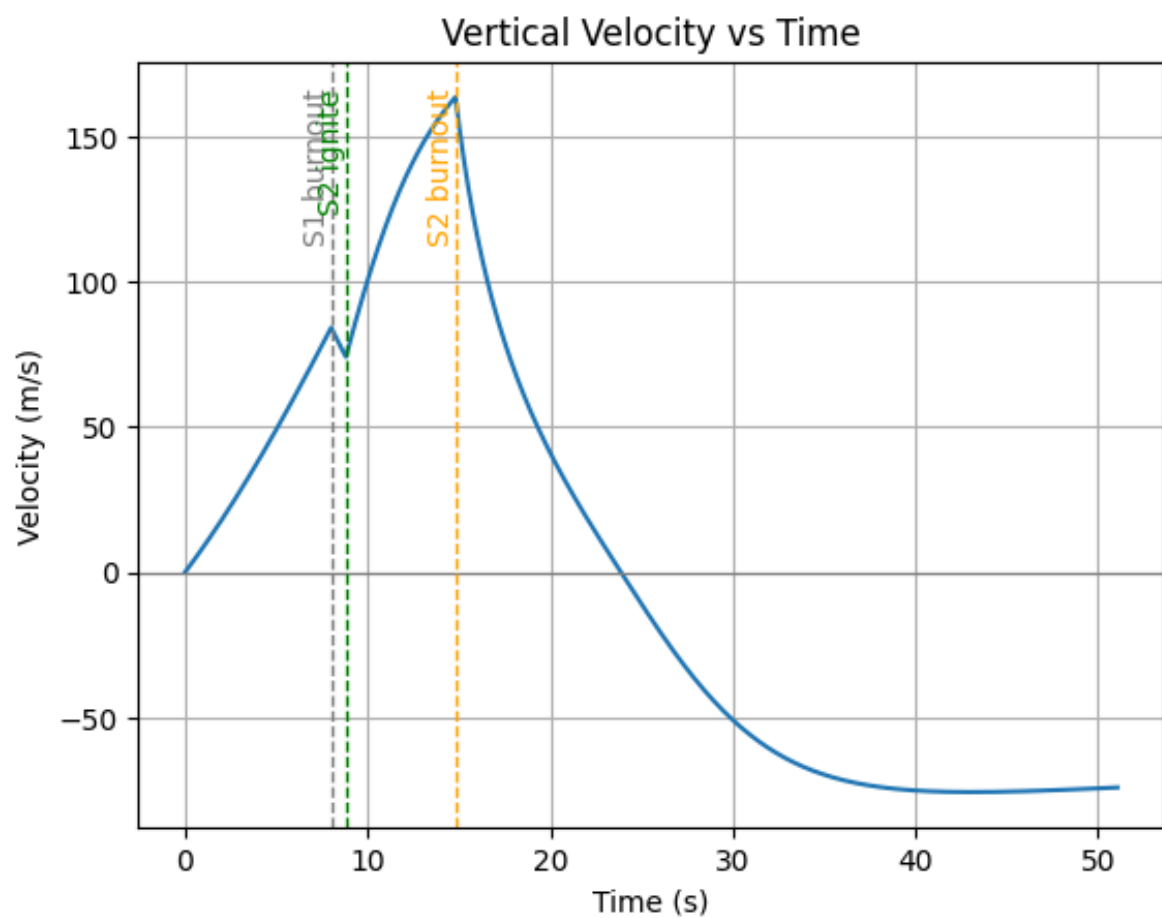
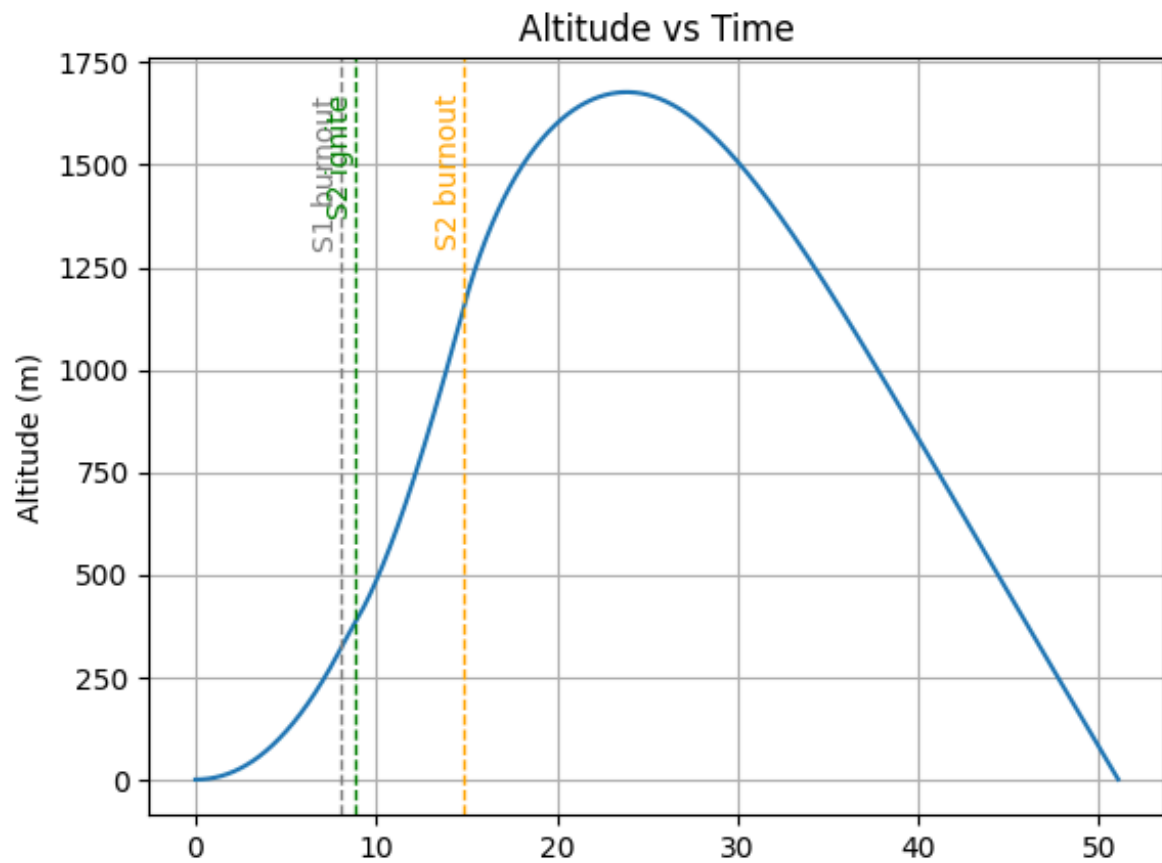
Mass Variation :

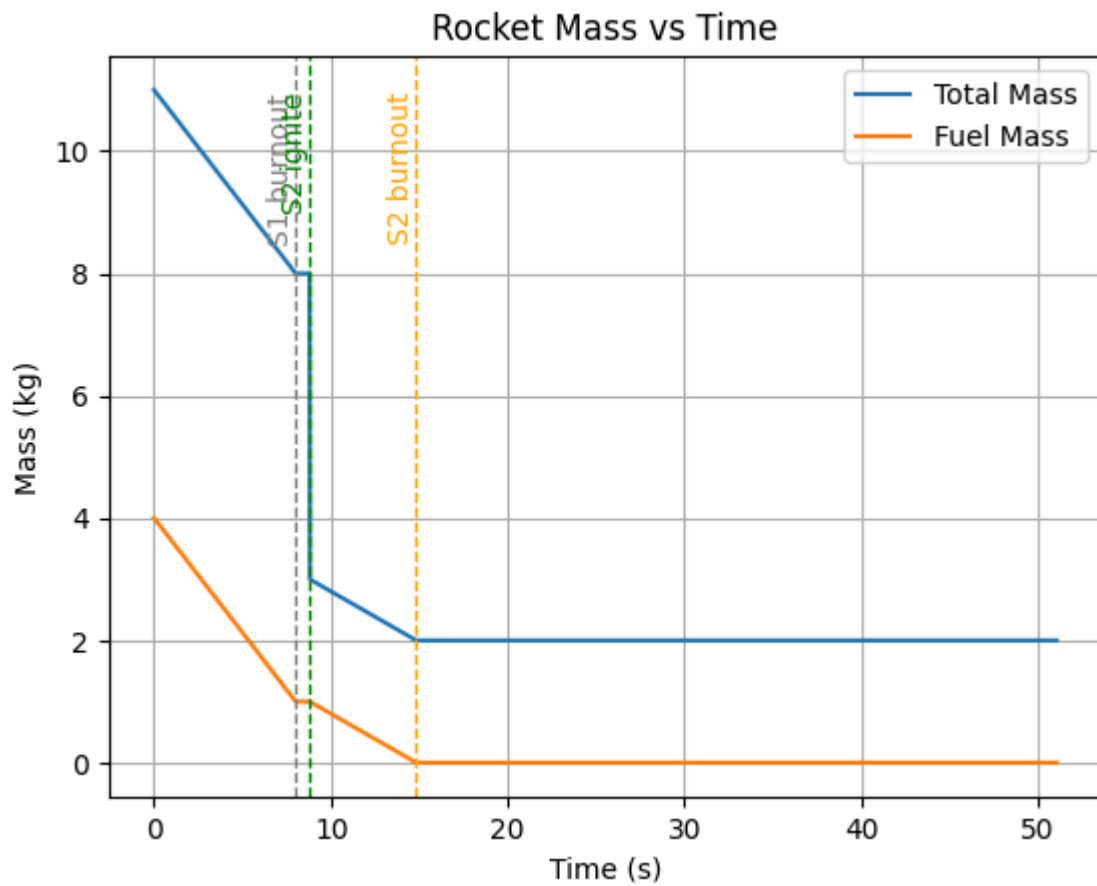
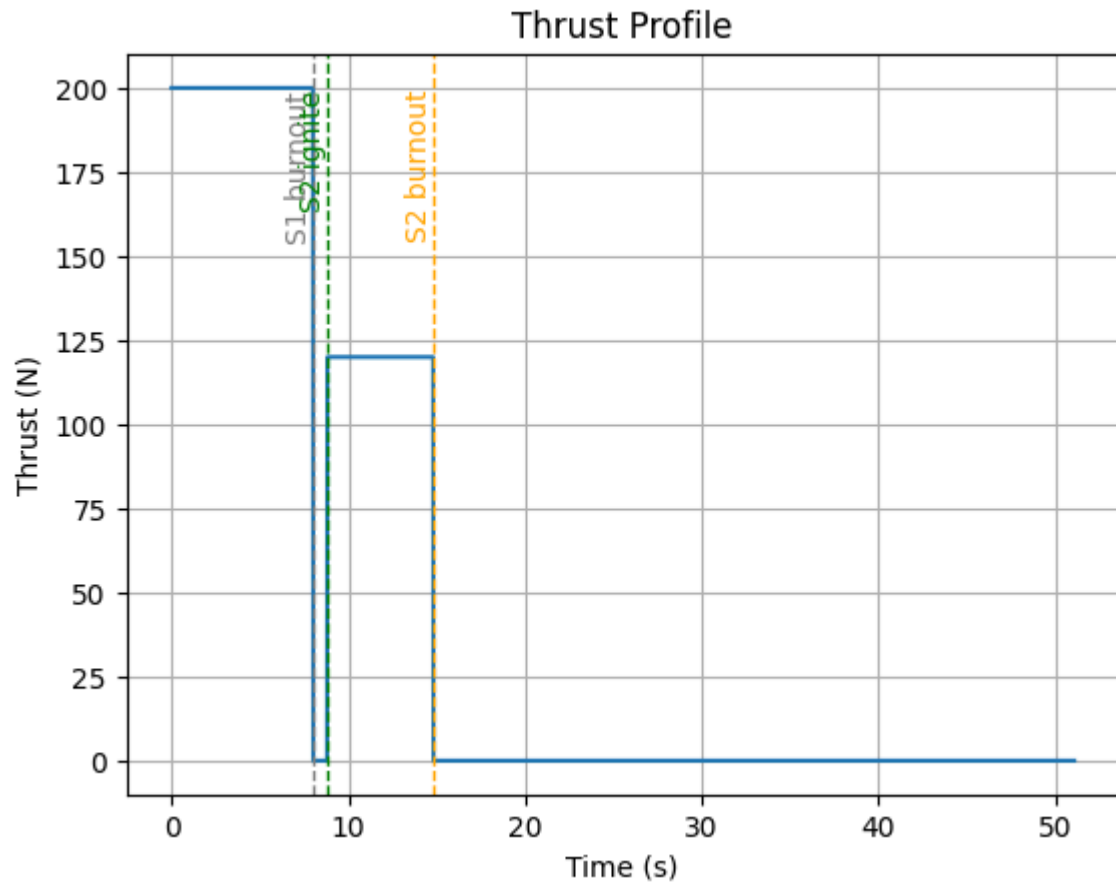
Before separation: $m(t) = \text{mass_dry}_1 + \text{mass_dry}_2 + \text{mass_fuel}(t)$

After separation: $m(t) = \text{mass_dry}_2 + \text{mass_fuel}(t)$

Condition of Auto-ignite :

$$q = \frac{1}{2} \cdot \rho(h) \cdot v^2 \leq 25 \text{ kPa} \quad , \Delta t \geq 0.8 \text{ s} \quad , v > 0$$





```

RocketSimulation — Bearbeitet

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from IPython.display import HTML

g, rho0, H = 9.81, 1.225, 8400.0
m_dry1, m_prop1, T1, burn1 = 5.0, 3.0, 200.0, 8.0
m_dry2, m_prop2, T2, burn2 = 2.0, 1.0, 120.0, 6.0
CD, A = 0.6, 0.01
dt, tmax = 0.01, 150.0
s0 = np.array([0.0, 0.0])
q_limit = 25000.0
min_coast = 0.8

def air_density(h):
    return rho0 * np.exp(-np.clip(h, 0, None)/H)

def dynamic_pressure(h, v):
    return 0.5 * air_density(h) * v**2

def thrust(t, h, v, state):
    if t <= burn1:
        return T1
    if state.get("ignite2") is None:
        if t >= burn1 + min_coast and dynamic_pressure(h, v) <= q_limit and v > 0:
            state["ignite2"] = t
            return 0.0
    if t <= state["ignite2"] + burn2:
        return T2
    return 0.0

def fuel_mass(t, state):
    ign2 = state.get("ignite2")
    if t <= burn1:
        return m_prop1*(1 - t/burn1) + m_prop2
    if ign2 is None or t <= ign2:
        return m_prop2
    if t <= ign2 + burn2:
        return m_prop2*(1 - (t - ign2)/burn2)
    return 0.0

def total_mass(t, state):
    ign2 = state.get("ignite2")
    if ign2 is None or t <= ign2:
        return (m_dry1 + m_dry2) + fuel_mass(t, state)
    return m_dry2 + fuel_mass(t, state)

def deriv(t, s, state):
    h, v = s
    m = total_mass(t, state)
    D = 0.5*air_density(h)*CD*A*v*abs(v)
    a = (thrust(t, h, v, state) - np.sign(v)*abs(D))/m - g
    return np.array([v, a])

def rk4_step(t, s, dt, state):
    k1 = deriv(t, s, state)
    k2 = deriv(t + 0.5*dt, s + 0.5*dt*k1, state)
    k3 = deriv(t + 0.5*dt, s + 0.5*dt*k2, state)
    k4 = deriv(t + dt, s + dt*k3, state)
    return s + (dt/6.0)*(k1 + 2*k2 + 2*k3 + k4)

t, s = 0.0, s0.copy()
state = {}
ts, hs, vs, ms, fs, Th = [t], [s[0]], [s[1]], [total_mass(t, state)], [fuel_mass(t, state)], [thrust(t, s[0], s[1], state)]

for _ in range(int(tmax/dt)):
    s = rk4_step(t, s, dt, state)
    t += dt
    if s[0] < 0 and t > 0.5:
        break
    ts.append(t)
    hs.append(max(s[0], 0.0))
    vs.append(s[1])
    ms.append(total_mass(t, state))
    fs.append(fuel_mass(t, state))
    Th.append(thrust(t, s[0], s[1], state))

t_touchdown = ts[-1]
t_burnout1 = burn1
t_ignite2 = state.get("ignite2", None)
t_burnout2 = (t_ignite2 + burn2) if t_ignite2 is not None else None
print(f"Stage-1 burnout: {t_burnout1:.2f}s")
print(f"Stage-2 ignition: {t_ignite2:.2f}s" if t_ignite2 is not None else "Stage-2 ignition: not triggered")
print(f"Stage-2 burnout: {t_burnout2:.2f}s" if t_burnout2 is not None else "Stage-2 burnout: n/a")
print(f"Touchdown: {t_touchdown:.2f}s Frames: {len(ts)}")

def mark(ax):
    if t_burnout1 is not None:
        ax.axvline(t_burnout1, ls='--', color='gray', lw=1)
        ax.text(t_burnout1, ax.get_ylim()[1]*0.95, 'S1 burnout', rotation=90, va='top', ha='right', color='gray')
    if t_ignite2 is not None:
        ax.axvline(t_ignite2, ls='--', color='green', lw=1)
        ax.text(t_ignite2, ax.get_ylim()[1]*0.95, 'S2 ignite', rotation=90, va='top', ha='right', color='green')
        ax.axvline(t_burnout2, ls='--', color='orange', lw=1)
        ax.text(t_burnout2, ax.get_ylim()[1]*0.95, 'S2 burnout', rotation=90, va='top', ha='right', color='orange')

plt.figure()
plt.plot(ts, hs)
plt.title("Altitude vs Time"); plt.xlabel("Time (s)"); plt.ylabel("Altitude (m)"); plt.grid(True); mark(plt.gca()); plt.show()

plt.figure()
plt.plot(ts, vs); plt.axhline(0, color='gray', lw=0.8)
plt.title("Vertical Velocity vs Time"); plt.xlabel("Time (s)"); plt.ylabel("Velocity (m/s)"); plt.grid(True); mark(plt.gca()); plt.show()

plt.figure()
plt.plot(ts, ms, label="Total Mass"); plt.plot(ts, fs, label="Fuel Mass")
plt.title("Rocket Mass vs Time"); plt.xlabel("Time (s)"); plt.ylabel("Mass (kg)"); plt.legend(); plt.grid(True); mark(plt.gca()); plt.show()

plt.figure()
plt.plot(ts, Th)
plt.title("Thrust Profile"); plt.xlabel("Time (s)"); plt.ylabel("Thrust (N)"); plt.grid(True); mark(plt.gca()); plt.show()

target_fps = 30
frame_step = max(1, int(round(1/dt)/target_fps))
frame_idx = range(0, len(ts), frame_step)

fig, ax = plt.subplots(figsize=(7,5))
ax.set_title("Two-Stage Rocket Flight (auto ignition)")
ax.set_xlabel("Time (s)")
ax.set_ylabel("Altitude (m)")
ax.set_xlim(0, ts[-1])
ax.set_ylim(0, max(hs)*1.1)
ax.grid(True)

(line,) = ax.plot([], [], lw=2, color='blue')
(point,) = ax.plot([], [], "ro", markersize=6)
txt = ax.text(0.98, 0.95, "", transform=ax.transAxes, fontsize=12,
             bbox=dict(facecolor='white', alpha=0.7, edgecolor='gray'),
             ha='right', va='top')

def init():
    line.set_data([], [])
    point.set_data([], [])
    txt.set_text("")
    return line, point, txt

def update(j):
    i = j
    line.set_data(ts[i:], hs[i:])
    point.set_data([ts[i-1]], [hs[i-1]])
    txt.set_text(f"t = {ts[i-1]:.1f} s\nFuel left: {fs[i-1]:.2f} kg\nThrust: {Th[i-1]:.0f} N")
    return line, point, txt

ani = FuncAnimation(fig, update, frames=frame_idx, init_func=init, interval=1000/target_fps, blit=True)
HTML(ani.to_html5_video())

```